

CENTRO UNIVERSITÁRIO FUNDAÇÃO ASSIS GURGACZ

ANDREI GUSTAVO DE LIMA MACEDO – RA: [202510047]

AURICELIA LIMA SOUSA – RA: [202510150]

CAROLINE BALKO PIOVESAM – RA: [202510088]

PEDRO HENRIQUE HIDEO MAKIYAMA – RA: [202510926]



**AVALIAÇÃO COMPARATIVA DE UCS, BUSCA GULOSA E A* NO PROBLEMA DO
LABIRINTO DE CAIXAS**

CASCADEL
SETEMBRO 2026

CENTRO UNIVERSITÁRIO FUNDAÇÃO ASSIS GURGACZ

ANDREI GUSTAVO DE LIMA MACEDO

AURICELIA LIMA SOUSA

CAROLINE BALKO PIOVESAM

PEDRO HENRIQUE HIDEO MAKIYAMA

**AValiação COMPARATIVA DE UCS, BUSCA GULOSA E A* NO PROBLEMA DO
LABIRINTO DE CAIXAS**

Este trabalho é parte integrante da avaliação bimestral da disciplina de Inteligência Artificial do 1º bimestre do curso de Engenharia de Software, 4º Período.

Professor(a): Marcos Augusto Campagnaro
Mucelini

Cascavel, 2026

Sumário

1. INTRODUÇÃO	4
2. MODELAGEM	4
3. HEURÍSTICA	5
4. RESULTADOS	6
5. DISCUSSÃO	6
6. CONCLUSÃO	10
7. REFERÊNCIAS	11

1. INTRODUÇÃO

Este relatório detalha a implementação e a comparação de algoritmos de busca aplicados ao problema 8, que simula um labirinto onde um agente precisa empurrar caixas até locais específicos marcados como alvos. A escolha deste problema decorre da complexidade do seu espaço de estados, que cresce de forma dinâmica conforme o número de elementos aumenta. Essa característica torna o cenário ideal para avaliar o desempenho prático entre buscas cegas e buscas guiadas por heurísticas, evidenciando como cada abordagem lida com o esforço computacional e o tempo de processamento.

2. MODELAGEM

A representação do estado do problema integra de forma unificada a coordenada atual do agente e as coordenadas de todas as caixas presentes no labirinto. O uso dessa tupla estruturada evita inconsistências lógicas que ocorreriam caso apenas a posição individual do agente fosse monitorada.

As ações disponíveis no ambiente consistem em deslocar o agente uma célula para cima, para baixo, para a esquerda ou para a direita. Quando o deslocamento aponta para uma célula ocupada por uma caixa, aplica-se a regra do empurrão, que avança a caixa para a posição seguinte desde que essa nova célula esteja totalmente livre de obstáculos. Cada movimento executado possui custo unitário fixo de valor um.

Para garantir a eficiência computacional, o código utiliza tuplas e conjuntos congelados do tipo *frozenset* para armazenar os estados, as paredes e as caixas. A escolha por estruturas imutáveis ocorre porque os algoritmos de busca utilizam dicionários e conjuntos (sets) na memória para registrar os caminhos já percorridos. Na linguagem Python, apenas dados com identidade fixa e imutável (*hash*) são aceitos como chaves de controle nessas tabelas. Dessa forma, o uso da tupla atende a essa exigência técnica da linguagem, prevenindo alterações acidentais na configuração do labirinto e viabilizando a validação imediata dos nós já explorados pelo algoritmo. No código, essa estrutura foi definida da seguinte forma:

```
estado_inicial = (agente, frozenset(caixas))
```

As particularidades do Problema 8 exigem o tratamento de becos sem saída, situação em que uma caixa é empurrada para um canto sem alvo e fica presa de forma definitiva. Para contornar esse problema, a modelagem inclui verificações matemáticas e espaciais aplicadas a cada empurrão simulado pelo agente. Se as adjacências da caixa movimentada formar uma quina encostada em paredes perpendiculares (um bloqueio simultâneo nos eixos vertical e horizontal) ou se for empurrada contra outros obstáculos formando um bloco maciço e sem espaço de manobra de 2x2 posições, a ação é classificada como inválida.

```
If self._caixa_canto(novo_estado) or  
self._bloqueio_2x2(novo_estado):  
    return False # O estado morto é imediatamente  
descartado
```

Através dessa lógica, o algoritmo descarta sumariamente os estados mortos antes de expandir a árvore de busca, poupando a máquina de processar caminhos sem solução e economizando um tempo significativo de execução.

3. HEURÍSTICA

A heurística adotada para guiar os algoritmos de busca foi a soma da distância de Manhattan de cada caixa até a posição-alvo mais próxima.

```
menor_dist = min(abs(caixa.x - alvo.x) + abs(caixa.y -  
alvo.y))
```

Essa abordagem surgiu a partir de um relaxamento do problema original, criando um cenário hipotético com o mínimo de restrições possíveis. Esse relaxamento assume um ambiente ideal onde as paredes do labirinto não existem, as caixas não bloqueiam a passagem umas das outras e o agente não gasta movimentos adicionais para caminhar e se reposicionar antes de realizar um empurrão.

A heurística escolhida é classificada como admissível porque a estimativa de passos calculada por ela sempre será menor ou igual ao custo real exigido para resolver o mapa. Como a matemática reflete uma versão relaxada do problema original, ela calcula a estimativa de custo mais baixa possível. Na prática, ao ignorar os obstáculos físicos e os deslocamentos extras do próprio agente, o cálculo nunca vai superestimar o esforço necessário para concluir o objetivo.

4. RESULTADOS

Tabela 1: Resultados para o Mapa 1 (1 Caixas)

ALGORITMO	HEURÍSTICA	CUSTO	PASSOS	NÓS EXPANDIDOS	TEMPO (MS)
UCS	Não usa	9	9	158	1.57
GULOSA	Manhattan	9	9	17	0.80
A*	Manhattan	9	9	41	0.68

Tabela 2: Resultados para o Mapa 2 (2 Caixas)

ALGORITMO	HEURÍSTICA	CUSTO	PASSOS	NÓS EXPANDIDOS	TEMPO (MS)
UCS	Não usa	17	17	1439	28.03
GULOSA	Manhattan	25	25	89	1.23
A*	Manhattan	17	17	406	6.73

Tabela 3: Resultados para o Mapa 3 (3 Caixas)

ALGORITMO	HEURÍSTICA	CUSTO	PASSOS	NÓS EXPANDIDOS	TEMPO (MS)
UCS	Não usa	28	28	43727	734.24
GULOSA	Manhattan	29	29	88	2.53
A*	Manhattan	28	28	17626	352.49

A análise dos resultados evidencia claramente o comportamento distinto de cada algoritmo diante do aumento da complexidade dos mapas do labirinto. A Busca de Custo Uniforme (UCS), por operar como uma busca cega sem o auxílio de heurísticas, revelou-se extremamente exaustiva. Embora tenha garantido a otimalidade ao encontrar sempre o menor custo e número de passos, a UCS sofreu um crescimento exponencial na quantidade de nós expandidos, saltando de 158 nós no Mapa 1 para impressionantes 43.727 nós no Mapa 3, o que impactou diretamente o tempo de processamento.

Em contrapartida, a Busca Gulosa destacou-se pela altíssima velocidade e economia drástica de recursos computacionais, guiando-se estritamente pela heurística de distância de Manhattan. No entanto, essa abordagem evidenciou sua principal limitação teórica, ou seja, a perda de otimalidade. Nos Mapas 2 e 3, a Gulosa priorizou caminhos diretos que resultaram em soluções subótimas, entregando custos e passos superiores aos obtidos pela UCS e pelo A* (como no Mapa 2, onde gerou custo 25 frente ao ótimo de 17).

Por fim, o algoritmo A* consolidou-se como a solução mais equilibrada e eficiente do estudo. Combinando o custo acumulado com a estimativa heurística ($g(n) + h(n)$), o A* garantiu a mesma otimalidade e os menores custos da UCS, mas com uma redução expressiva de mais de 50% nos nós expandidos e no tempo de execução no cenário mais complexo. Esse contraste comprova empiricamente que o uso de uma heurística admissível direciona a busca com precisão, unindo a garantia de caminhos ótimos à eficiência computacional.

5. DISCUSSÃO

1. Algum algoritmo devolveu uma solução pior que os outros? Qual, e por quê?

Sim, a Busca Gulosa produziu caminhos subótimos nos Mapas 2 e 3. Enquanto a UCS e o algoritmo A* encontraram rotas perfeitas com custos de 17 e 28, a Gulosa finalizou os mesmos mapas com custos de 25 e 29, respectivamente. Esse comportamento ocorre porque a Busca Gulosa toma decisões baseando-se exclusivamente na heurística, ou seja, no palpite de quanto falta para o objetivo ($h(n)$). Ao ignorar o custo real das ações já realizadas ($g(n)$), o algoritmo sofre de uma visão míope, escolhendo atalhos aparentes que muitas vezes levam a caminhos consideravelmente mais longos.

2. Qual expandiu menos nós? Isso significa que ele é o melhor?

A Busca Gulosa foi a que expandiu o menor número de nós em todos os cenários testados, precisando abrir apenas 17 nós no Mapa 1, contra 158 da UCS. No entanto, essa economia extrema de processamento não faz dela o melhor algoritmo, já que esse ganho de velocidade cobra um preço alto na qualidade da solução, gerando trajetos ineficientes nos mapas maiores. O algoritmo A* demonstrou ser a opção superior e mais equilibrada do projeto, pois reduz drasticamente a quantidade de nós gerados em relação à busca cega, mantendo, ainda assim, a garantia matemática de sempre encontrar a rota ótima.

3. A heurística de vocês é admissível? Como sabemos?

Sim, a heurística utilizando a distância de Manhattan é estritamente admissível. Temos essa certeza porque ela calcula a distância direta de cada caixa até o seu respectivo alvo relaxando regras fundamentais do labirinto, como a presença de paredes impenetráveis, a colisão com outras caixas e os passos gastos pelo próprio agente para se posicionar. Como a ausência desses obstáculos físicos facilita o percurso e encurta a distância aparente, o valor calculado pela estimativa será sempre menor ou igual à quantidade real de passos exigidos, cumprindo a regra de nunca superestimar o verdadeiro custo da solução.

4. Se vocês relaxassem mais regras do problema, a heurística ficaria melhor ou pior? Por quê?

A heurística ficaria pior, pois perderia grande parte do seu poder de orientação. Quando relaxamos regras em excesso, o valor estimado cai drasticamente e se distancia da realidade do labirinto, resultando em uma heurística fraca. Ao receber estimativas muito baixas e irreais, o algoritmo A* passaria a desconfiar menos dos caminhos ruins e seria forçado a investigar quase todas as direções disponíveis. Na prática, isso faria o A* perder sua precisão característica e se degradar, assumindo o comportamento exaustivo, lento e ineficiente de uma busca cega.

5. Houve empate entre algoritmos? Se houve, o que isso revela sobre o problema?

Sim, ocorreu um empate técnico completo no primeiro mapa (1 Caixa), onde os três algoritmos finalizaram a resolução com um custo exato de 9 passos. Esse comportamento revela que, em instâncias com espaços muito pequenos e abertos, a restrição espacial é baixa o suficiente para que a intuição imprecisa da Busca Gulosa accidentalmente coincida com o caminho ótimo. Além disso, a UCS e o A* empataram no custo da solução em todos os mapas testados, revelando que ambos possuem a mesma garantia de otimalidade teórica, divergindo apenas na quantidade de esforço computacional necessário para encontrar a saída.

6. Em que situação real vocês usariam cada um dos três?

A Busca de Custo Uniforme é indicada para cenários onde encontrar a rota perfeita é absolutamente inegociável e o ambiente do problema é pequeno o suficiente para não sobrecarregar a memória do sistema. A Busca Gulosa, por sua vez, seria a melhor escolha para aplicações onde a resposta precisa ser imediata, como a movimentação de dezenas de

personagens figurantes em um videogame, situação em que um caminho rápido e aproximado é preferível à exatidão. Por fim, o algoritmo A* seria o eleito para sistemas modernos de rotas por GPS e navegação de robôs autônomos, pois atua no mundo real entregando o melhor percurso possível sem desperdiçar processamento do hardware.

6. CONCLUSÃO

O desenvolvimento deste projeto permitiu compreender na prática as vantagens e as limitações das diferentes abordagens de inteligência artificial aplicadas ao problema 8. Os resultados comprovaram que a busca cega possui um alto custo de processamento e que as estratégias guiadas por heurísticas trazem saltos enormes de eficiência computacional. O algoritmo A* provou ser a ferramenta mais robusta e aplicável para desafios reais, unindo a velocidade de exploração de rotas com a garantia matemática de sempre encontrar a solução mais barata.

Além das métricas de desempenho, o maior aprendizado do trabalho está na própria arquitetura do código implementado. Durante a programação ficou evidente que a Busca de Custo Uniforme, a Busca Gulosa e o A* são exatamente o mesmo algoritmo executado por uma única função de busca. O que muda o comportamento da inteligência artificial é apenas a regra matemática escolhida para organizar a fronteira de estados. Compreender que uma simples troca na equação de avaliação transforma uma busca exaustiva em uma navegação rápida e inteligente é a grande lição que consolida toda a teoria estudada.

7. REFERÊNCIAS

RED BLOB GAMES. **Introduction to A***. Disponível em:
<https://www.redblobgames.com/pathfinding/a-star/introduction.html>. Acesso em: 12 set. 2026.

JSOKO. **Sokoban Deadlocks**. Disponível em:
<https://jsokoapplet.sourceforge.io/sokoban/deadlocks.html>. Acesso em: 12 set. 2026.

MUCELINI, Marcos Augusto Campagnaro. **Agentes Inteligentes e Resolução de Problemas de Busca**. Material didático (Slides de aula), 2026.

MUCELINI, Marcos Augusto Campagnaro. **Problema de Busca e Busca Não Informada (Cega)**. Material didático (Slides de aula), 2026.

MUCELINI, Marcos Augusto Campagnaro. **Busca informada: Best first, Gulosa e A***. Material didático (Slides de aula), 2026.